**CPSC 320 Intermediate Algorithm Design and Analysis.**
**Professor: Abner Turkieltaub**

# Homework 2

This assignment is due **Friday, October 9 at 23:59 PM**. Late submissions will be accepted until Sunday October 11 at 23:59, but penalized: -5 points for Saturday and -15 for Sunday submissions. Please follow these guidelines:

- Prepare your solution using LaTeX and submit a pdf file. Easiest will be to submit using the .tex file provided. For questions where you need to select an option, simply change `\square` (□) to `\blacksquare` (■).
- Start your solution after our `\color{blue}` command. Your solution should appear in blue, making it a lot easier for TAs to find what you wrote.
- Start each problem on a new page, using the same numbering and ordering as this assignment handout. You can use `\newpage` to start a new page.
- Submit the assignment through GradeScope. Your group must make a **single** submission via one group member's account, marking all other group members in that submission **using GradeScope's interface**.
- When uploading to Gradescope, link each question with the page(s) of your pdf containing your solution.

Before we begin, a few notes on pseudocode throughout CPSC 320: Your pseudocode should communicate your algorithm clearly, concisely, correctly, and without irrelevant detail. Reasonable use of plain English is fine in such pseudocode. You should envision your audience as a capable CPSC 320 student unfamiliar with the problem you are solving. You may **neither** include what we consider to be irrelevant coding details **nor** assume that we understand any particular programming language[1].

Remember also to **justify/explain your answers**. We understand that gauging how much justification to provide can be tricky. Inevitably, judgment is applied by both student and grader as to how much is enough, or too much, and it's frustrating for all concerned when judgments don't align. Justifications/explanations need not be long or formal, but should be clear and specific (referring back to lines of pseudocode, for example). Proofs should be a bit more formal.

On the plus side, if you choose an incorrect answer when selecting an option but your reasoning shows partial understanding, you might get more marks than if no justification is provided. And the effort you put in writing down the justification will hopefully help you gain deeper understanding and may well help you converge to the right selection :).

Ok, time to get started...

[1] So, for example, do not write `#include <iostream>` at the start of your pseudocode, and avoid language-specific notation like C/C++/Java's ternary (question-mark-colon) operator.

# 1 Slow as a logarithm.

In "Algorithm Design" by Kleinberg and Tardos you can find (2.8, page 41) the following useful fact, stating roughly that logarithmic functions are upper-bounded by polynomial functions, more precisely:

**Fact:** for every $b > 1$ and every $x > 0$, we have $\log_b n \in O(n^x)$.

a) (10 pts.) Use the fact above to prove the following stronger assertion:

For every $b > 1$ and every $x > 0$, we have $\log_b n \in o(n^x)$.

b) (10 pts.) Use part a) to show that $\sqrt{n} \in o(n/\log^{12} n)$. The log is base 10, and $\log^k n = (\log n)^k$.

## 2 I wanna be the very best

In the game "Pokemon Go", there is a pvp[2] feature. In this feature, each player selects a team of 3 different pokemons to fight a match against another player. Assume that each pokemon can have one of three roles: "Lead", "Safe swap" or "Closer". We say that a team is *balanced* if it has the three roles[3].

a) (5 pts.) We say that a valid team is just a set of three different pokemons. If you have $n$ pokemons to choose from, how many valid teams are there?

b) (10 pts.) We want to print a list with all the balanced teams. Consider a brute force approach where you look at every valid team and you print it if it is balanced. Write a pseudo-code to implement this idea using a nested for loop structure, and find a tight $O$-bound for the worst case runtime. Your input is an array of pokemon $[p_1, p_2, \ldots, p_n]$ and you can query the role of any pokemon using $p_i$.role() in constant time. Justify your runtime.

c) (5 pts.) Usually brute force is too slow, but in this case it is not that bad. Assuming that printing one team takes constant time, show that there are instances for which any algorithm that must print all balanced teams has $\Omega(n^3)$ runtime.

d) (5 pts.) Define the following sets:

$$L = \{p_i : p_i.\text{role}() = \text{Lead}\}$$
$$S = \{p_i : p_i.\text{role}() = \text{Safe Swap}\}$$
$$C = \{p_i : p_i.\text{role}() = \text{Closer}\}$$

Write the number of balanced teams as a function of $|L|,\ |S|,\ |R|$.

e) (10 pts.) In general, you express the runtime as a function of the input size. Sometimes you can analyze the runtime in terms of other parameters to have a more detailed answer. First analyze the runtime of your previous brute force approach in terms of $|L|,\ |S|$ and $|R|$. Then describe a more efficient algorithm in terms of $|L|,\ |S|$ and $|R|$. For your second algorithm, what is the best and worst possible runtime in terms of $n$?

[2] Player vs player.

[3] In case you are curious, you start a match with 2 shields that can be used to block attacks. The lead should be strong at the beginning of the match when both players have shields. The save swap should be versatile and have few hard counters, so when the lead is unfavourable, you can switch to it. The closer is strong at the end of the game when shields are down. All of this is irrelevant to answer the question.

## 3 Stirling

The Stirling approximation is a famous and useful formula that approximates the factorial. One way to express it is as follows:

$$\ln(n!) = n \ln n - n + O(\ln n).$$

In other words, $\ln(n!)$ is approximately equal to $n \ln n - n$ plus something bounded by a logarithmic term. In particular, we can say that $\ln(n!) \in \Theta(n \ln n)$. In this question we will prove this simplified approximation (without using Stirling).

a) (5 pts.) Prove that $\ln(n!) \in O(n \ln n)$.

b) (10 pts.) Using induction, prove that $(n!)^2 \geq n^n \ \forall n \in \mathbb{N}$ .
*Hint:* it may be useful to prove by induction the following property first: $\forall n \in \mathbb{N},\ n^n \geq (n+1)^{n-1}$.

c) (10 pts.) Use the previous result to conclude that $\ln(n!) \in \Theta(n \ln n)$

d) (5 pts.) Either prove or provide a counterexample for the following statement: if $h$ is an increasing function and $h(f(n)) \in \Theta(h(g(n)))$, then $f(n) \in \Theta(g(n))$.

## 4 It's not the destination, it's the journey.

In planet Namek, all the alien kids have now their school for the upcoming school year. For parents, the question now is what is the fastest way to get to school. For parents with only one kid this is trivial since they can use noodle maps to find the shortest path from their home to the school[4]. Consider a parent of twins in different schools, this parent wants to find the shortest route to drop both kids and then return home[5].

a) (5 pts.) For each of the following scenarios, explain how they can find the shortest route using noodle maps the least number of times possible.

- Case 1: distances are symmetric (distance from $A$ to $B$ is the same as the distance from $B$ to $A$).
- Case 2: distances are asymmetric[6].

b) (5 pts.) Many algorithms for shortest path (like Dijkstra) actually find the shortest path from one vertex to all the others. Imagine you are a developer for noodle maps and you want to implement a query where the user (a parent) inputs their home and the two schools and it shows them the shortest route starting from their home, passing by the two schools and returning home. Assume that as a developer, when you run noodle maps to find a shortest path from $A$ to $B$, you can actually access the shortest paths from $A$ to every other location. Answer the questions from the previous part under this new hypothesis.

c) (5 pts.) Answer the questions from the previous two parts, but now under the assumption that work from home is not allowed, so parents must go to their work after dropping both kids.

[4] In noodle map you can enter two locations $A$ and $B$ and it tells you the shortest path from $A$ to $B$.

[5] In Namek, all parent can work from home.

[6] Maybe there's a school in the top of a mountain and you have to take some long stairs to get there, but then when you leave you can go down a slide.

## Statement on Collaboration and Use of Resources

To develop good practices in doing homeworks, citing resources and acknowledging input from others, please complete the following.

None of your answers here will be used against you, so please be honest. Not answering this question will result in a deduction of 2 points.

1. Please list the CWLs of all group members.
2. List here any books or online sources that you consulted and that led you to the solution of one or more problems (indicate the problem(s) after each resource):
3. One or more of us consulted with course staff during office hours:
   ☐ Yes ☐ No
4. One or more of us collaborated with other CPSC 320 students outside our group:
   ☐ Yes ☐ No
   If yes, please list their name(s) here:
5. One or more of us used an AI to solve one or more questions from this assignment:
   ☐ Yes ☐ No
   If yes, please say which problem(s) was(were) solved using AI and which AI model you used:
6. One or more of us collaborated with or consulted others outside of CPSC 320:
   ☐ Yes ☐ No
   If yes, please list their name(s) here: